

Docker — Documentation

Présentation

L'environnement de développement d'ETNAir repose entièrement sur **Docker Compose**, ce qui permet à n'importe quel développeur de démarrer le projet complet en une seule commande, sans avoir à installer manuellement Node.js, PostgreSQL ou PgAdmin sur sa machine. Cette approche garantit aussi que tous les contributeurs travaillent avec exactement les mêmes versions des dépendances et des services, ce qui élimine les classiques "ça marche chez moi".

Cinq services tournent en parallèle sur un réseau Docker isolé. Le **frontend** est exposé sur le port 5173 et sert l'application Vue via le dev server de Vite. L'**API** Express tourne sur le port 3000. La base **PostgreSQL** écoute sur le port 5432 (mappé en 5433 côté hôte pour éviter les conflits avec une éventuelle instance locale). Une instance **PgAdmin** est disponible sur le port 5050 pour explorer la base via une interface graphique. Tous ces services sont reliés par un réseau bridge nommé `etnair-network`, ce qui leur permet de communiquer entre eux en utilisant leurs noms de service (`api`, `db`, etc.) plutôt que des adresses IP.

Structure du docker-compose.yml

Le fichier `docker-compose.yml` à la racine du projet déclare l'ensemble des services. Pour chaque service, on définit son image ou son build, ses ports exposés, ses variables d'environnement, ses volumes et son éventuel `depends_on` qui indique qu'il doit démarrer après un autre.

Le service `db` utilise l'image officielle `postgres:16` et déclare trois variables : `POSTGRES_USER`, `POSTGRES_PASSWORD` et `POSTGRES_DB`, qui sont automatiquement consommées par l'image pour créer la base et le compte au premier démarrage. Un volume nommé `db_data` est monté sur `/var/lib/postgresql/data` pour que les données persistent entre les redémarrages.

Le service `api` est construit à partir du `Dockerfile` situé dans `./api`. Il déclare son `DATABASE_URL` complet, son `JWT_SECRET`, et `PORT`. La directive `depends_on: [db]` garantit que PostgreSQL démarre avant l'API. Un bind mount `./api/uploads:/app/uploads` permet aux images uploadées d'être visibles depuis l'hôte, ce qui est pratique pour les inspecter mais surtout pour qu'elles survivent aux rebuilds de l'image. La politique `restart: unless-stopped` redémarre automatiquement le conteneur en cas de crash.

Le service `frontend` est lui aussi buildé depuis son `Dockerfile`. Il reçoit la variable `API_URL=http://api:3000` qui est utilisée par le proxy Vite pour rediriger les appels `/api/*` vers le backend. Un bind mount `./frontend:/app` permet le hot reload pendant le développement : toute modification de fichier sur l'hôte est immédiatement visible dans le conteneur. Un volume anonyme sur `/app/node_modules` est crucial car il empêche le bind mount d'écraser les `node_modules` installés dans l'image (qui sont parfois différents selon la plateforme).

Enfin le service `pgadmin` utilise l'image officielle avec `harvey@etnair.xyz` comme email par défaut et `harvey` comme mot de passe.

Commandes courantes

Pour **démarrer toute la stack**, il suffit d'une seule commande à la racine du projet : `docker compose up -d`. Le flag `-d` (detached) lance les services en arrière-plan. Au premier lancement, Docker doit télécharger les images et builder l'API et le frontend, ce qui prend quelques minutes. Les lancements suivants sont quasi instantanés.

Pour **voir l'état de tous les conteneurs**, on utilise `docker compose ps`. Pour **suivre les logs** d'un service en temps réel, c'est `docker compose logs -f api` (ou `frontend`, ou `db`). Cette commande est très utile pour

diagnostiquer un démarrage qui se passe mal.

Quand on modifie le `Dockerfile` ou le `package.json` d'un service, il faut **reconstruire l'image** avec `docker compose build api`, puis relancer le service avec `docker compose up -d api`. À l'inverse, si on modifie juste du code source (et qu'on a un bind mount), pas besoin de rebuild : le hot reload prend le relais.

Pour **tout reset**, y compris la base de données, la commande est `docker compose down -v`. Le `-v` supprime aussi les volumes, ce qui efface complètement la base. Cette commande est utile quand on veut tester le seed depuis zéro ou quand la base est dans un état corrompu.

Enfin, pour **exécuter une commande dans un conteneur** sans avoir à entrer dedans, on utilise `docker compose exec`. Par exemple `docker compose exec api npm test` lance les tests Jest. `docker compose exec api npx prisma migrate deploy` applique les migrations. `docker compose exec db psql -U etnair_user -d etnair_db` ouvre une console PostgreSQL.

Dockerfile de l'API

Le `Dockerfile` de l'API repose sur l'image `node:24-alpine` qui est légère (environ 150 Mo) et stable. Le workflow est classique : on définit le répertoire de travail `/app`, on copie les fichiers de package, on installe les dépendances de production uniquement (`npm ci --omit=dev`), on copie le schéma Prisma puis on génère le client Prisma, et enfin on copie le reste du code.

Le `docker-entrypoint.sh` est copié dans `/usr/local/bin` et rendu exécutable. Il est ensuite défini comme `ENTRYPOINT` du conteneur, ce qui signifie qu'il est toujours exécuté en premier. Sa responsabilité est d'appliquer les migrations Prisma, de lancer le seed si nécessaire, puis de démarrer le serveur Node.

⚠ **Attention** : le script `docker-entrypoint.sh` doit absolument être enregistré avec des fins de ligne LF (Linux) et pas CRLF (Windows). Sous VS Code sur Windows, il faut vérifier le sélecteur en bas à droite de la fenêtre et basculer en LF si nécessaire. Sinon, le shell Alpine renvoie une erreur cryptique du type `exec ./docker-entrypoint.sh: no such file or directory`. Le fichier `.gitattributes` à la racine du projet force désormais les fins de ligne LF pour tous les `.sh`.

Dockerfile du frontend

Le frontend utilise aussi `node:24-alpine`. Pour le développement local, l'image lance simplement `npm run dev -- --host`, ce qui démarre Vite avec l'option `--host` pour qu'il écoute sur toutes les interfaces (pas seulement 127.0.0.1), condition indispensable pour qu'on puisse y accéder depuis l'hôte.

En production (image utilisée par Kubernetes), un build statique est généré avec `npm run build`, et les fichiers sont servis par un nginx léger plutôt que par le dev server Vite. Cette approche est beaucoup plus performante en production et permet le caching agressif des assets.

Réseau interne et résolution DNS

Tous les conteneurs sont placés dans le même réseau Docker bridge `etnair-network`. Docker fournit automatiquement une résolution DNS interne entre eux, basée sur le nom des services. Concrètement, depuis le conteneur frontend, on peut faire une requête vers `http://api:3000` et Docker route automatiquement vers le bon conteneur. C'est ce mécanisme que le proxy Vite exploite avec sa variable `API_URL`.

De la même manière, l'API se connecte à PostgreSQL via `db:5432` (et non `localhost:5432`), et PgAdmin utilise aussi `db:5432` comme host dans sa configuration. Cette approche évite tout couplage avec des adresses IP qui

peuvent changer.

Persistance des données

Trois types de données doivent persister entre les redémarrages. Pour la **base PostgreSQL**, on utilise un volume nommé `db_data`. Tant qu'on ne fait pas `docker compose down -v`, ce volume reste intact même si on supprime et recrée le conteneur.

Pour les **images uploadées** par les propriétaires, on utilise un bind mount qui pointe vers `./api/uploads` sur l'hôte. Cela a deux avantages : les images sont accessibles directement depuis le système de fichiers de l'hôte (pratique pour les inspecter ou les sauvegarder), et elles survivent même à un rebuild complet de l'image API.

Pour les **node_modules du frontend**, on utilise un volume anonyme. C'est une astuce classique : le bind mount `./frontend:/app` écraserait sinon les `node_modules` du conteneur. En montant un volume anonyme par-dessus le sous-chemin `/app/node_modules`, on garde les `node_modules` installés dans l'image tout en bénéficiant du hot reload sur le reste du code.

Healthcheck et résilience

L'API expose un endpoint `GET /health` qui retourne `{ "status": "OK" }`. Cet endpoint sert à plusieurs choses : en Docker Compose, il pourrait être utilisé pour un healthcheck déclaré dans le compose, et en Kubernetes il est utilisé pour les probes liveness et readiness. Combiné avec la directive `restart: unless-stopped`, cela rend l'API auto-réparante : si elle crash, Docker la redémarre automatiquement, et le healthcheck garantit qu'elle est prête à servir avant de recevoir du trafic.

Problèmes courants et solutions

Plusieurs erreurs reviennent fréquemment lors du setup initial. La première est le **CRLF dans docker-entrypoint.sh** déjà mentionné, qui se manifeste par un `no such file or directory` au démarrage de l'API. La solution est de convertir le fichier en LF.

Une autre erreur classique est le **frontend qui ne charge pas correctement quand on l'accède via un reverse proxy HTTPS** (par exemple celui d'une école). Le hot reload WebSocket de Vite ne fonctionne pas bien dans ce cas. La solution est d'accéder directement à `http://localhost:5173` plutôt qu'à l'URL proxifiée.

Une troisième erreur survient quand l'API tente de démarrer avant que PostgreSQL ne soit prêt à accepter des connexions. Bien que `depends_on` garantisse l'ordre de démarrage, il n'attend pas que le service soit "ready". Le retry intégré à Prisma résout généralement ce problème en quelques secondes, mais on peut aussi ajouter un healthcheck PostgreSQL et conditionner le démarrage de l'API dessus.

Enfin, quand on déploie le frontend derrière un proxy HTTPS, il faut hardcoder la `baseURL` Axios à `/api` (sans préfixe `http://`) sinon on a des erreurs de Mixed Content. Le proxy Vite se charge de relayer vers le bon backend.